The *while* Loop

Section 3

Chapter 3

An Introduction to Programming Using Python
David I. Schneider

Loops

- Topics
  - While loops
  - *Break* and *continue* statements

Relational and Logical Operators

- A *condition* is an expression
  - Involving relational operators (such as < and >=)
  - Logical operators (such as and, or, and not)
  - Evaluates to either True or False
- Conditions used to make decisions
  - Control loops
  - Choose between options

The *while* Loop

- Executes a block of code repeatedly
- *while* loop repeatedly executes an **indented** block of statements
  - As long as a certain condition is met
- Form

```
while condition:
    indented block of statements
```

- Continuation condition is a boolean expression

The *while* Loop

- Example 1: Program displays 1 – 5, after loop terminates, *num* will be 6

```
## Display the numbers from 1 to 5.
num = 1
while num <= 5:
    print(num)
    num += 1   # Increase the value of num by 1.
```

[Run]

```
1
2
3
4
5
```

While Loop

– Example:

```
x, sum = 0,0
while x < 10:
        x += 2
        sum += x
print(sum, x)
```

Initial: x, sum = 0, 0
Iteration 1: x, sum = 2, 2
Iteration 2: x, sum = 4, 6
Iteration 3: x, sum = 6, 12
Iteration 4: x, sum = 8, 20
Iteration 5: x, sum = 10, 30

What is final value of sum and x ?
Answer: sum=30, x=10

The *while* Loop

- Example 2: Input validation.

```
## Movie Quotations
print("This program displays a famous movie quotation.")
responses = ('1', '2', '3')
response = '0'
while response not in responses:
    response = input("Enter 1, 2, or, 3: ")
    if response == '1':
       print("Plastics.")
    elif response == '2':
        print("Rosebud.")
    elif response == '3':
        print("That's all folks.")
```

[Run]

```
This program displays a famous movie quotation.
Enter a whole number from 1 to 3: one
Enter a whole number from 1 to 3: 5
Enter a whole number from 1 to 3: 2
Rosebud.
```

The *break* Statement

- Causes an exit from anywhere in the body of a loop
- When **`break`** is executed
  - Loop immediately terminates
- Break statements usually occur in *if* statements

The *break* Statement

- Example 6: Program uses *break* to avoid two input statements.

```
## Obtain list of numbers.
list1 = []
while True:
    num = eval(input("Enter a nonnegative number: "))
    if num == -1:
        break   # Immediately terminate the loop.
    list1.append(num)
```

Your Turn …

- *Break*: exit from current loop

  - Example:

```
x, sum = 0,0
while x < 10:
        x += 2
        if x%3 == 0:
            break
        sum += x
        print(sum, x)
```

Initial: x, sum = 0, 0
Iteration 1: x, sum = 2, 2
Iteration 2: x, sum = 4, 6
Iteration 3: x, sum = 6, -

Answer: 6, 6

The *continue* Statement

- When **`continue`** executed in a while loop
  - Current iteration of the loop terminates
  - Execution returns to the loop's header
- Usually appear inside *if* statements.

The *continue* Statement

- Example 7: Searches a list for the first *int* object that is divisible by 11.

```
## Find integers divisible by 11.
list1 = ["one", 23, 17.5, "two", 33, 22.1, 242, "three"]
i = 0
foundFlag = False
while i < len(list1):
    x = list1[i]
    i += 1
    if not isinstance(x, int):
        continue   # Skip to next item in list.
    if x % 11 == 0:
        foundFlag = True
        print(x, "is the first int that is divisible by 11.")
        break
if not foundFlag:
    print("There is no int in the list that is divisible by 11.")
```

[Run]

```
33 is the first int in the list that is divisible by 11.
```

Your Turn …

- *Continue*: control returns to top of current loop
- *Break*: exit from current loop
  - Example:

```
x, sum = 0,0
while x < 10:
        x += 2
        if x%3 == 0:
            continue
        sum += x
        print(sum, x)
```

Initial: x, sum = 0, 0
Iteration 1: x, sum = 2, 2
Iteration 2: x, sum = 4, 6
Iteration 3: x, sum = 6, 6
Iteration 4: x, sum = 8, 14
Iteration 5: x, sum = 10, 24

Answer: 24, 10

Infinite Loops

- Example 9: Condition ***number >= 0*** always true.

```
## Infinite loop.
print("(Enter -1 to terminate entering numbers.)")
number = 0
while number >= 0:
    number = eval(input("Enter a number to square: "))
    number = number * number
    print(number)
```

End

Section 3

Chapter 3

An Introduction to Programming Using Python
David I. Schneider